



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

**«Челябинский государственный университет»
(ФГБОУ ВО «ЧелГУ»)**

Институт информационных технологий

Кафедра информационных технологий и экономической информатики

ОТЧЕТ по лабораторной работе №1

В лабораторной работе приняли участие:

Черепанов Марк Даниилович
Куныгин Сергей Александрович
Шахмухаметова Анна Павловна

Преподаватель: Николаев Иван Евгеньевич

1. Цель работы

Экспериментально исследовать временную сложность алгоритмов, сравнить фактическое время с теоретической оценкой $T(n) = C \cdot f(n)$, построить аппроксимацию и оценить её качество (MSE), освоить методику корректного бенчмаркинга.

2. Постановка задачи

Разработано приложение с выбором алгоритма, конфигурацией, БД результатов с кэшем, графиками, аппроксимацией $C \cdot f(n)$ и MSE. Дополнительно реализован отдельный операционный бенчмарк возведения в степень ($n = 1..1000$), в котором измеряется количество операций, а не время.

3. Теоретические основы

- **Сложности:** $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$.
- **Теоретическое время:** $T(n) = \sum_i \text{count}_i(n) \cdot \text{cost}_i$; константы cost_i определяются калибровкой.
- **Аппроксимация:** $T_{\text{approx}}(n) = C \cdot f(n)$, $C = \sum f \cdot t / \sum f^2$ (МНК через начало координат).
- **MSE** = $(1/k) \cdot \sum (T_{\text{эмп}}(n_i) - T_{\text{approx}}(n_i))^2$.

4. Описание алгоритмов

Формат раздела: описание → идея → псевдокод → теоретическая сложность (Best/Average/Worst) → количество элементарных операций → практическое применение → преимущества → недостатки.

Состав: №1–№9 — стандартные алгоритмы временного бенчмарка; №10–№12 — индивидуальные (кастомные) алгоритмы. Отдельно — операционный бенчмарк возведения в степень (приложение в конце документа): $n = 1..1000$, измеряется количество операций.

№1. Постоянная функция — $O(1)$ (ConstantAlgorithm)

Описание. Эталон алгоритма постоянной сложности: количество операций не зависит от размера задачи.

Идея. Логическая задача $O(1)$ выполняется за ~ 1 нс — меньше разрешения Stopwatch. Чтобы время было измеримым, фиксированная операция (целочисленное сложение)

технически повторяется n раз. n здесь — число повторений замера, а не размер задачи: это явно отделено в коде и в интерфейсе.

Псевдокод.

```
асс ← 0 для i от 1 до n:      # технические повторения      асс ← асс + 7
```

Сложность. $O(1)$ логически; $T(n) = C \cdot n$ относительно числа повторений.

Количество операций. На итерацию: 2 сложения (асс и счётчик), 1 сравнение, 1 присваивание.

Применение. Доступ по индексу, арифметика, обмен переменных.

Преимущества. Мгновенно, не зависит от данных.

Недостатки. Как учебный замер требует повторений; чувствителен к конвейеру процессора.

№2. Сумма элементов массива — $O(n)$ (SumAlgorithm)

Описание. Один проход по массиву с накоплением суммы в 64-битной переменной.

Идея. Каждый элемент обрабатывается константное время → суммарно линейно.

Псевдокод.

```
sum ← 0
для каждого x в массиве:
    sum ← sum + x
```

Сложность. $O(n)$, Best = Average = Worst.

Количество операций. На элемент: 2 сложения, 1 сравнение, 1 присваивание, 1 доступ к массиву.

Применение. Агрегации, средние значения, контрольные суммы.

Преимущества. Простота, последовательный доступ.

Недостатки. JIT может векторизовать цикл (SIMD) — реальное время лучше «элементарной» модели; для больших n время ограничено пропускной способностью памяти.

№3. Произведение элементов — $O(n)$ (ProductAlgorithm)

Описание. Один проход с накоплением произведения по модулю простого числа $P = 2^{31} - 1$ — защита от переполнения без изменения сложности.

Псевдокод.

```
prod ← 1 для каждого x в  
массиве:      prod ← (prod ·  
x) mod P
```

Сложность. $O(n)$.

Количество операций. На элемент: 2 умножения (умножение + остаток), 1 сложение, 1 сравнение, 2 присваивания, 1 доступ.

Применение. Полиномиальные хэши, модульная арифметика.

Преимущества. Контролируемое переполнение.

Недостатки. Остаток от деления дорог.

№4а. Многочлен наивно — $O(n^2)$ (PolynomialNaiveAlgorithm)

Описание. $P(x) = a_0 + a_1 \cdot x + \dots + a_n \cdot x^n$; для каждого слагаемого степень x вычисляется заново внутренним циклом.

Псевдокод.

```
result ← 0 для i от  
0 до n:      xk ← 1  
для j от 1 до i:  
xk ← xk · x  
result ← result + a[i] · xk
```

Сложность. $O(n^2)$: $\sum i \approx n^2/2$ умножений.

Количество операций. $\approx n^2/2$ умножений и $n^2/2$ сложений.

Применение. Учебный «анти-пример»; черновые вычисления малых степеней.

Преимущества. Наглядно показывает цену дублирования работы.

Недостатки. Квадратичный рост.

№46. Метод Горнера — $O(n)$ (HornerAlgorithm)

Описание. $P(x) = (...((a_n \cdot x + a_{n-1})) \cdot x + ...) \cdot x + a_0$ — вложенные скобки устраняют повторное возведение в степень.

Псевдокод.

`result ← a[n] для i от n-1 вниз до 0:      result ← result · x + a[i]`

Сложность. $O(n)$: n умножений и n сложений.

Количество операций. На коэффициент: 1 умножение, 1 сложение, 1 сравнение, 1 присваивание, 1 доступ.

Применение. Быстрые вычисления многочленов, полиномиальные хэши, системы счисления.

Преимущества. Минимум умножений; контраст с №4а.

Недостатки. Последовательная зависимость итераций.

№5а. Итеративное возведение в степень — $O(n)$ (PowerIterativeTimeAlgorithm)

Описание. `result ← 1`; затем n раз `result ← result · a mod m` — ровно n умножений.

Идея. Лобовое умножение; время линейно по показателю. Так как один вызов занимает микросекунды, за один замер алгоритм выполняется $K = 1000$ раз (технические повторения, как у №1). Показатель n — размер задачи.

Псевдокод.

`result ← 1 для i
от 1 до n:
 result ← result · a mod m`

Сложность. Best = Average = Worst = $O(n)$.

Количество операций. $\approx 5n$ (умножение+остаток, сравнение, присваивания, инкремент), $\times K$.

Применение. Учебный базовый вариант.

Преимущества. Простейший код.

Недостатки. Линейный рост; для сравнения с №5в даёт в ~50 раз больше операций при $n = 1000$.

№5б. Рекурсивное возведение в степень — $O(n)$ (PowerRecursiveTimeAlgorithm)

Описание. $\text{power}(a, k) = a \cdot \text{power}(a, k-1)$, $\text{power}(a, 0) = 1$ — рекурсия глубиной n .

Псевдокод.

```
power(a, k):  
    если k = 0: вернуть 1  
    вернуть a · power(a, k-1) mod m
```

Сложность. $O(n)$ по операциям и по глубине стека (показатель ограничен 5000 — безопасно для стека 1 МБ).

Количество операций. $\approx 6n$ (умножения, сравнения, вычитания), $\times K = 20000$.

Применение. Демонстрация рекурсии.

Преимущества. Наглядность.

Недостатки. Глубина стека $O(n)$, накладные расходы вызовов.

№5в. Бинарное возведение в степень — $O(\log n)$ (PowerBinaryTimeAlgorithm)

Описание. Двоичное разложение показателя: основание возводится в квадрат на каждом шаге, при единичном бите результат домножается.

Псевдокод.

```
result ← 1; b ← a; e ← n пока e > 0:  
    если e нечётно: result ← result · b mod m  
    b ← b · b mod m  
    e ← e >> 1
```

Сложность. Best = Average = Worst = $O(\log n)$: $\approx \log_2(n)$ итераций.

Количество операций. $\approx 6 \cdot \log_2(n)$, $\times K = 1\,000\,000$.

Применение. RSA, Диффи–Хеллман, биномиальные коэффициенты по модулю.

Преимущества. На $n = 10^9$ — ~ 30 итераций вместо 10^9 .

Недостатки. Рост $\log n$ слабо заметен на линейной шкале (диапазон подобран так, чтобы рост был виден).

№6. Bubble Sort — $O(n^2)$ (BubbleSortAlgorithm)

Описание. Проходы по массиву с обменом соседних элементов; флаг `swapped` даёт ранний выход (лучший случай $O(n)$ на отсортированных данных).

Псевдокод.

```
для end от n-1 вниз до 1:
    swapped ← false    для j от
0 до end-1:           если a[j] >
a[j+1]:               обменять
a[j], a[j+1]          swapped
← true               если not swapped:
прервать
```

Сложность. Best $O(n)$ / Average $O(n^2)$ / Worst $O(n^2)$.

Количество операций. Сравнений $n(n-1)/2$; обменов в среднем $n(n-1)/4$ (инверсии); доступов $\approx 2 \cdot C + 4 \cdot S$.

Применение. Обучение, почти отсортированные малые данные.

Преимущества. Простейший код, стабильность.

Недостатки. Квадратичное время.

№7. Quick Sort — $O(n \log n)$ (QuickSortAlgorithm)

Описание. Собственная реализация: медиана трёх, разделение по Седжвику, рекурсия по меньшей части, вставки для отрезков < 16 . Не вызов `Array.Sort`.

Псевдокод.

```
sort(lo, hi):    если hi-lo+1 < 16: insertionSort(lo,
hi); вернуть    p ← partition(lo, hi)        # медиана
трёх
                sort(lo, p-1); sort(p+1, hi)
```

Сложность. Best/Average $O(n \log n)$, Worst $O(n^2)$ (редок на случайных данных).

Количество операций. Сравнения $\approx 1.39 \cdot n \cdot \log_2 n$; обмены $\approx 0.25 \cdot n \cdot \log_2 n$.

Применение. Универсальная сортировка (`std::sort`, `Arrays.sort`).

Преимущества. Быстрая, in-place.

Недостатки. Неустойчива; возможен худший случай; память до 10^8 элементов — 400 МБ.

№8. TimSort — $O(n \log n)$ (TimSortAlgorithm)

Описание. Собственная упрощённая реализация: естественные серии (с разворотом убывающих), вставки до `minrun` (32..63), стек серий с инвариантами баланса, стабильные слияния через буфер меньшей стороны.

Псевдокод.

```
minrun ← calcMinRun(n) пока есть элементы:      run ←  
естественная серия с i (убывающая → разворот)    если  
короче minrun: дотянуть вставками      push run;  
mergeCollapse() слить оставшиеся серии
```

Сложность. Best $O(n)$ (отсортированные данные), Average/Worst $O(n \log n)$.

Количество операций. $\approx n \cdot \log_2 n$ сравнений, $2 \cdot n \cdot \log_2 n$ перемещений.

Применение. Стандарт Python (`list.sort`) и Java (`Object[]`).

Преимущества. Стабильность, адаптивность.

Недостатки. Сложнее; $O(n/2)$ буфера.

№9. Классическое умножение матриц — $O(n^3)$ (MatrixMultiplicationAlgorithm)

Описание. $C = A \cdot B$ тройным вложенным циклом: $C[i,j] = \sum A[i,k] \cdot B[k,j]$.

Идея. Для каждой пары (i, j) суммируются n произведений — n^3 умножений и n^3 сложений. Порядок циклов i - k - j — последовательное чтение строки B (кэш).

Псевдокод.

для i, k, j :
 $c[i,j] \leftarrow c[i,j] + a[i,k] \cdot b[k,j]$

Сложность. Best = Average = Worst = $O(n^3)$.

Количество операций. n^3 умножений + n^3 сложений + $\sim 3n^3$ накладных + $\sim 4n^3$ доступов.

Применение. Графика, СЛАУ, графы, базовые операции нейросетей.

Преимущества. Простой эталон кубической сложности.

Недостатки. Проигрывает Штрассену и BLAS на больших n ; память $3 \cdot n^2$ элементов.

№10. Индивидуальный: быстрое возведение в степень — $O(\log n)$ (CustomAlgorithm)

Описание. $a^n \bmod m$ двоичным возведением (то же ядро, что №5в, но замер без адаптера-дубликата: $K = 200\,000$ повторений на замер). Образец кастомного алгоритма студента.

Псевдокод.

```
result ← 1; b ← a mod m; e ← n пока e > 0:
если e нечётно: result ← result·b mod m
b ← b·b mod m    e ← e >> 1
```

Сложность. Best = Average = Worst = $O(\log n)$.

Количество операций. $\approx 3 \cdot \log_2(n)$ умножений на итерацию (квадрат+остаток, домножение при бите), $\times K$.

Применение. RSA, Диффи–Хеллман, модульная арифметика.

Преимущества. Логарифмический рост, нет больших массивов.

Недостатки. Рост слабо заметен на линейной шкале.

№11. Индивидуальный: алгоритм Кадане — $O(n)$ (KadaneAlgorithm)

Описание. Максимальная сумма непрерывного подмассива за один проход.

Идея. `maxEnding` — лучшая сумма, заканчивающаяся в позиции i (продолжить предыдущий подмассив или начать новый); `maxSoFar` — глобальный рекорд. Один проход, $O(1)$ памяти.

Псевдокод.

```
maxEnding  $\leftarrow$  0; maxSoFar  $\leftarrow$   $-\infty$  для каждого  
x в массиве:    maxEnding  $\leftarrow$  max(x,  
maxEnding + x)    maxSoFar  $\leftarrow$   
max(maxSoFar, maxEnding)
```

Сложность. Best = Average = Worst = $O(n)$.

Количество операций. На элемент: 3 сложения, 3 сравнения, 2 присваивания, 1 доступ к массиву.

Применение. Максимальная прибыль по дням, анализ временных рядов, биоинформатика.

Преимущества. Один проход, константная память, ветвящийся (нетривиальный) поток.

Недостатки. Ветвления дороже линейных «поточковых» операций из-за промахов предсказателя. Данные содержат отрицательные значения — иначе задача вырождается.

№12. Индивидуальный: реверс массива — $O(n)$ (ReverseArrayAlgorithm)

Описание. Оборот массива на месте двумя указателями: обмен $a[i] \leftrightarrow a[n-1-i]$, указатели идут от краёв к центру.

Псевдокод.

```
i  $\leftarrow$  0; j  $\leftarrow$  n-1 пока i <  
j:    обменять a[i],  
a[j]    i  $\leftarrow$  i + 1; j  $\leftarrow$   
j - 1
```

Сложность. Best = Average = Worst = $O(n)$: $n/2$ обменов, $O(1)$ памяти.

Количество операций. На элемент: 0.5 обмена, 1 сложение, 0.5 сравнения, 2 доступа к массиву.

Применение. Обработка строк и буферов, разворот истории, подготовка данных для стека.

Преимущества. Минимум дополнительной памяти, симметричный доступ к памяти.

Недостатки. Запись по двум фронтам массива; повторные проходы чередуют порядок (объём работы на проход одинаков).

Приложение. Операционный бенчмарк возведения в степень ($n = 1..1000$)

Отдельный режим (кнопка «Возведение в степень»): измеряется НЕ время, а количество элементарных операций — счётчик инкрементируется внутри алгоритма. График: $X = n$, $Y = \text{операций}$; три кривые на одном графике.

Итеративный — $O(n)$ (PowerIterativeAlgorithm)

$\text{result} \leftarrow 1$; n раз $\text{result} \leftarrow \text{result} \cdot a \bmod m$. Измеренно: $\approx 5n$ операций (умножение+остаток, присваивание, сравнение, инкремент).

Рекурсивный — $O(n)$ (PowerRecursiveAlgorithm)

$\text{power}(a, k) = a \cdot \text{power}(a, k-1)$. Измеренно: $\approx 6n$ операций (плюс вызовы).

Бинарный — $O(\log n)$ (PowerBinaryAlgorithm)

Двоичное разложение показателя. Измеренно: $\approx 5 \cdot \log_2(n) + 3 \cdot \text{popcount}(n)$ операций. Для $n = 1000$: итеративный ≈ 5000 операций, бинарный ≈ 80 — разница $\sim 60x$.

5. Архитектура приложения

Приложение построено по паттерну **MVVM** (WPF, net9.0-windows):

- **Views/MainWindow.xaml** — привязки к ViewModels/MainViewModel, ChartControl, экспорт PNG.
- **Controls/ChartControl** — собственный график.
- **Services:**
 - **BenchmarkService** — основная процедура измерения времени выполнения алгоритма (бенчмарк);

- `OperationBenchmarkService` — операционный бенчмарк возведения в степень;
- `StatisticsService` — статистика (среднее, min, max, σ);
- `ApproximationService` — аппроксимация $C \cdot f(n)$, MSE, R^2 ;
- `CalibrationService` — калибровка констант стоимости;
- `DataPreparationService` — подготовка данных (вне таймера);
- `ExportService` — экспорт CSV/JSON, PNG;
- `BenchmarkDatabase` (SQLite) — хранение и кэш.
- **Models(POCO):** `BenchmarkConfiguration`, `BenchmarkResult/StatisticsResult`, `OperationCounts/OperationCostModel`, `OperationsPoint`, `ExperimentReport`, `ChartData`.
- **Algorithms:** `AlgorithmBase` (абстрактный) ← 10 алгоритмов временного бенчмарка; `AlgorithmRegistry` (единая точка регистрации); `IOperationCountedAlgorithm` ← 3 алгоритма возведения в степень.

Схема БД (SQLite):

Таблица `Experiments`: `Id`, `ConfigHash`, `SchemaVersion`, `AlgorithmName`, `Complexity`, `StartN`, `NMax`, `StepN`, `RunsPerPoint`, `Seed`, `CostAddition`, `CostMultiplication`, `CostComparison`, `CostAssignment`, `CostSwap`, `CostArrayAccess`, `MSE`, `ApproximationC`, `Conclusion`, `CreatedUtc`.

Таблица `Measurements`: `Id`, `ExperimentId` (FK → `Experiments.Id` ON DELETE CASCADE), `N`, `RunNumber`, `ElapsedTimeNs`, `StepCount`, `TheoreticalNs`, `ExperimentDate`.

Индексы: `IX_Experiments_Hash(ConfigHash)`, `IX_Measurements_HashN(ExperimentId, N, RunNumber)`.

Логика кэша

Ключ кэша — SHA-256 от: алгоритм + `ComplexityClass` + `StartN` + `Nmax` + `step` + запусков + `seed` + `PhysicalBlockLimit` + константы стоимости + тег режима («TIME» или «OPS»). При включённом «Use cache» точка с уже измеренным ключом берётся из БД. «Force recalculation» удаляет все данные с этим ключом перед запуском — старые и новые результаты не смешиваются. После калибровки констант хэш меняется, старые результаты не считаются подходящими.

Поток выполнения эксперимента:

1. `MainViewModel.RunExperimentAsync` — валидация полей, запуск в `Task.Run`, `IProgress<BenchmarkProgress>` + `CancellationToken`.
2. `BenchmarkService.RunAsync`: хэш конфигурации; `Force recalculation` → удаление старых данных; JIT-прогрев; для каждой точки: кэш БД ИЛИ измерение

(данные вне таймера, прогревочный запуск, отдельные запуски → Stopwatch → БД); лимит времени точки (PointTimeoutException).

3. `ApproximationService.Fit` — кривая $T_{approx}(n) = C \cdot f(n)$, MSE, R^2 .

4. `MainViewModel` — серии графика, MSE в GUI, авторский вывод, итоги (MSE, C) в БД.

5. Экспорт CSV/JSON, график → PNG.

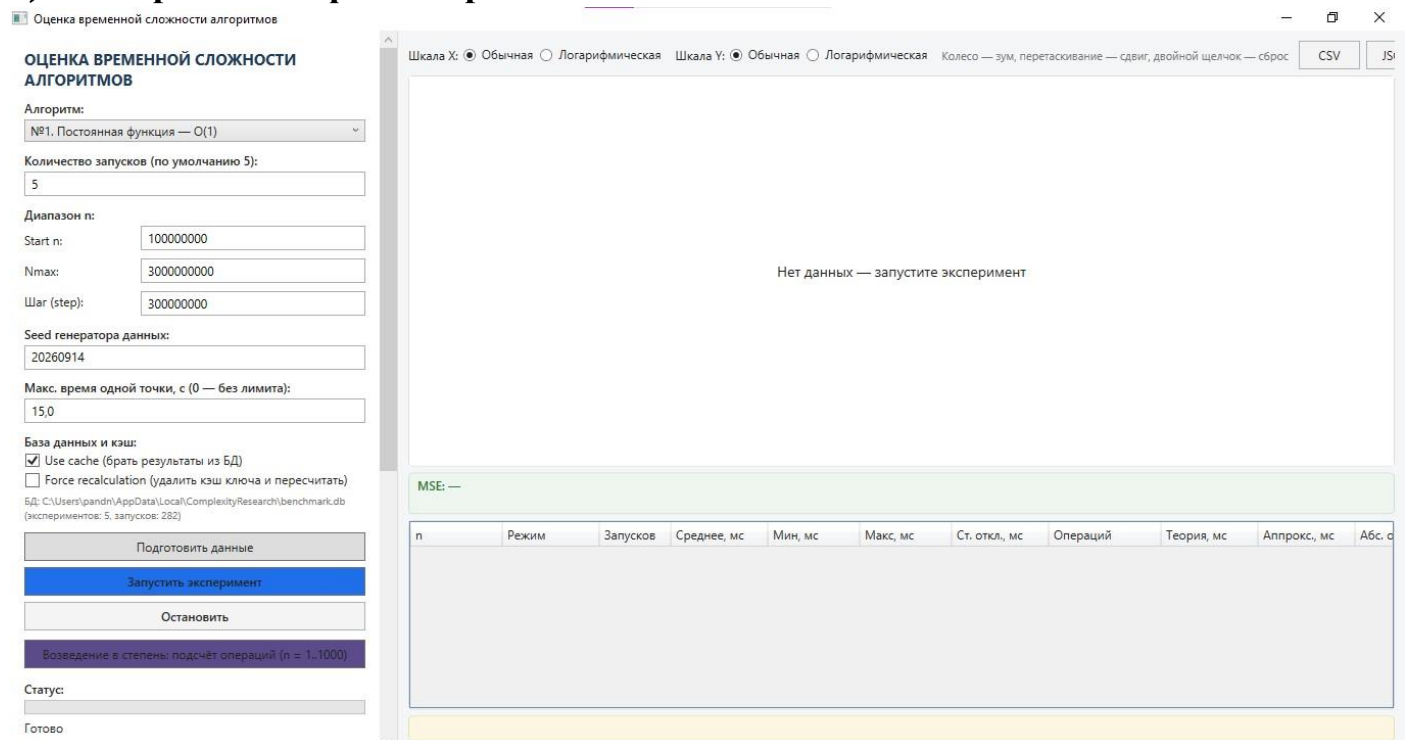
Принцип расширения: новый алгоритм = класс-наследник `AlgorithmBase` + одна строка в `AlgorithmRegistry`. Сервисы, график, таблица, БД и экспорт не меняются (принцип открытости/закрытости).

Потокобезопасность: эксперимент — в фоновом потоке; UI-обновления — через `Progress<T>` и `Dispatcher.BeginInvoke`; SQLite — короткоживущие соединения, пул отключён, обращения только из фонового потока.

Тесты: `ComplexityResearch.Tests` (xunit): корректность алгоритмов и сортировок, матрицы, многочлены (naïve = Horner = `Math.Pow`), возведение в степень (результат + счётчик операций), аппроксимация/MSE, БД/кэш, генерация точек, интеграция бенчмарк+БД, отмена.

Руководство по приложению:

1) Открыть .exe файл с приложением



2) Выбрать интересующий алгоритм из выпадающего списка слева вверху

Алгоритм:

№1. Постоянная функция — $O(1)$

Алгоритм:

№1. Постоянная функция — $O(1)$

№1. Постоянная функция — $O(1)$

№2. Сумма элементов — $O(n)$

№3. Произведение элементов — $O(n)$

№4а. Многочлен (наивно) — $O(n^2)$

- 3) Далее устанавливаем количество запусков, начальный $n(\text{start } n)$, конечный $n(n \text{ max})$, шаг(step) и другие далее описанные

Диапазон n :

Start n : 100000000

Nmax: 1000000000

Шаг (step): 100000000

Seed генератора данных:

20260914

Макс. время одной точки, с (0 — без лимита):

15,0

База данных и кэш:

☒ Use cache (брать результаты из БД)

☐ Force recalculation (удалить кэш ключа и пересчитать)

БД: C:\Users\pandn\AppData\Local\ComplexityResearch\benchmark.db

(экспериментов: 5, запусков: 282)

- 4) Производим калибровку констант

Калибровка констант

Константа сложения: 0,500 нс
Константа умножения: 1,000 нс
Константа сравнения: 0,500 нс
Константа присваивания: 0,300 нс
Константа обмена: 1,500 нс
Константа доступа к массиву: 1,000 нс
(значения по умолчанию (учебные))

- 5) Можем ознакомиться с описанием алгоритма

О выбранном алгоритме

Сложность: $O(n)$

Последовательный проход по массиву с накоплением произведения по модулю простого числа (переполнение контролируется модульной арифметикой). Для больших n — масштабированный режим.

Применение: Вычисление вероятностей совместных событий, хэш-функции (полиномиальные хэши), модульная арифметика в криптографии.

Формула теоретического времени:

$T(n) = n \cdot (2 \cdot C_{\text{умн}} + C_{\text{слож}} + C_{\text{срав}} + 2 \cdot C_{\text{присв}} + C_{\text{дост}})$ — умножение, остаток ($\approx$ умножение), инкремент, сравнение, присваивания и доступ к массиву на элемент

- 6) Выбираем особенности графика и ознакомливаемся с тем, как можно лучше рассмотреть график

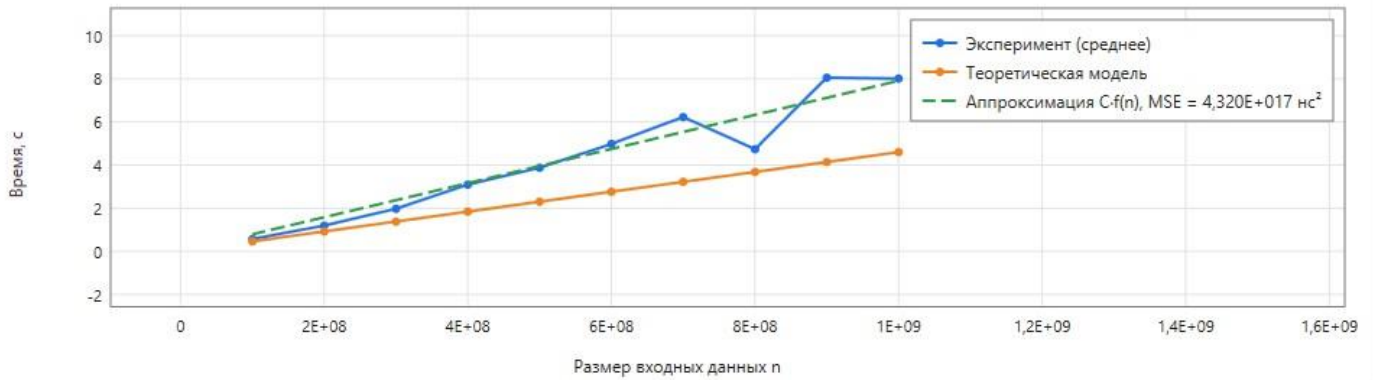
Шкала X: ☒ Обычная ☐ Логарифмическая Шкала Y: ☒ Обычная ☐ Логарифмическая Колесо — зум, перетаскивание — сдвиг, двойной щелчок — сброс

- 7) Подготавливаем данные и запускаем. В случае чего останавливаем:

Подготовить данные
Запустить эксперимент
Остановить
Возведение в степень: подсчёт операций ($n = 1..1000$)

- 8) Получаем разного рода данные

№3. Произведение элементов — $O(n)$ — $MSE = 4,320E+017 \text{ нс}^2$



$MSE = 4,320E+017 \text{ нс}^2$ — $R^2 = 0,9313$
 $T_{\text{approx}}(n) = 7,910 \cdot n$ ($R^2 = 0,9313$)

n	Режим	Запусков	Среднее, мс	Мин, мс	Макс, мс	Ст. откл., мс	Операций	Теория, мс	Аппрокс., мс
100,000,000	Масштабированный	5	568.0669	499.5806	656.6828	51.7760	700,000,000	460.0000	790.9631
200,000,000	Масштабированный	5	1190.8535	1074.4729	1348.2205	104.8178	1,400,000,000	920.0000	1581.9262
300,000,000	Масштабированный	5	1971.3708	1698.1880	2207.6660	219.8599	2,100,000,000	1380.0000	2372.8893
400,000,000	Масштабированный	5	3092.8345	2922.4042	3436.7366	178.9649	2,800,000,000	1840.0000	3163.8524
500,000,000	Масштабированный	5	3875.4786	3708.9052	4105.4024	155.7520	3,500,000,000	2300.0000	0.0000
600,000,000	Масштабированный	5	4976.8219	3670.6873	6112.4489	917.1131	4,200,000,000	2760.0000	0.0000

Теоретическая сложность: $O(n)$

Эксперимент показал:
 при увеличении n время возрастает приблизительно как $n^{1,17}$ (теоретический показатель: 1,0).

Аппроксимация: $T_{\text{approx}}(n) = 7,910 \cdot n$ ($R^2 = 0,9313$)
 $MSE = 4,320E+017 \text{ нс}^2$ (средний квадрат отклонения эксперимента от кривой $Cf(n)$; низкий MSE подтверждает форму роста, но не является абсолютным доказательством модели).
 Средняя относительная ошибка (эксперимент vs теоретическая модель): 60,3%

9) Если нужно конвертируем в нужный формат



6. Методика эксперимента

Что измеряется: только выполнение ядра алгоритма (ExecuteCore). НЕ измеряются: генерация данных, выделение массивов/матриц и буферов, БД, графики, GUI, экспорт, вывод на экран. Подготовка данных выполняется до запуска таймера (PrepareInput / PrepareRunInput — вне Stopwatch).

Таймер и окружение:

- `System.Diagnostics.Stopwatch` (счётчик высокого разрешения), не `DateTime.Now`;
- конфигурация **Release** (в Debug JIT не оптимизирует — время искажается);
- JIT-прогрев: весь алгоритм прогоняется на малом n до серии; первый запуск каждой точки — прогревочный;
- защита от оптимизации: результат алгоритма «утекает» в статическое поле (`PreventOptimization`);
- в измеряемом коде нет аллокаций → GC не влияет; между точками — явная сборка мусора;

- входные данные одинаковые для всех запусков точки (seed фиксирован); сортировки получают свежую копию данных на каждый запуск (вне таймера).

Параметры эксперимента:

Параметр	Значение по умолчанию	Комментарий
StartN	свой алгоритма	первая точка
Nmax	свой алгоритма	последняя точка, включается всегда
step	свой алгоритма	линейная сетка n
запусков на точку	5	настраивается; сверх них 1 прогревочный запуск
Seed	20260914	воспроизводимость данных
лимит времени точки	15 с	превышение → корректная остановка эксперимента

Эмпирический подбор Nmax:

Алгоритм	Nmax	Время макс. точки (ориентир)
Константа O(1)	$3 \cdot 10^9$	~1–2 с
Сумма O(n)	$3 \cdot 10^9$	~2 с (память ~12 ГБ/с)
Произведение O(n)	10^9	~5 с (остаток от деления дорог)
Многочлен наивно	50 000	~1.5 с ($n^2/2$ умножений)
Горнер	$1.5 \cdot 10^9$	~6 с
Bubble Sort	50 000	~2–3 с
Quick Sort	10^8	~10 с (400 МБ массив)
TimSort	$7 \cdot 10^7$	~9 с (280 МБ массив)

Умножение матриц	600	~2–5 с ($2 \cdot 600^3$ операций)
Индивидуальный	100 000	~10 мс (K повторений на замер)

Дополнительно `MaxSecondsPerPoint` страхует от «зависания» при ручном увеличении `Nmax`.

Прямой и масштабированный режимы:

- **Прямой** ($n \leq 50\,000\,000$): массив размера n реально выделяется.
- **Масштабированный** (линейные алгоритмы, $n > 50$ млн): обрабатывается блок 50 млн элементов, число проходов пропорционально логическому n . Суммарный объём работы пропорционален n , но массив размером n не выделяется. Такие точки помечаются «Масштабированный» и не выдаются за полное физическое измерение. Сортировки всегда прямые ($n \log n$ не декомпозируется линейно).

Статистика: по $K = 5+$ учитываемым запускам каждой точки: среднее, `min`, `max`, стандартное отклонение; отдельные запуски хранятся в БД (по строке на запуск).

Возведение в степень — особый режим: для итеративного/рекурсивного/бинарного возведения ($n = 1..1000$) измеряется не время, а количество элементарных операций. График: $X = n$, $Y = \text{операций}$.

7. Описание элементарных операций

Таблица констант после калибровки. **Константы стоимости являются изменяемыми в приложении:** они задаются в `OperationCostModel`, могут быть переопределены через `CalibrationService` (автоматическая калибровка) или вручную в интерфейсе. После изменения констант хэш конфигурации меняется, и старые результаты кэша не считаются подходящими.

Пример:

Операция	Обозначение	Значение(нс/операция)	Способ изменения
Сложение	<code>C_слож</code>	0,050	<code>OperationCostModel.CostAddition</code> / калибровка / вручную
Умножение	<code>C_умн</code>	0,050	<code>OperationCostModel.CostMultiplication</code> / калибровка / вручную
Сравнение	<code>C_срав</code>	0,050	<code>OperationCostModel.CostComparison</code> / калибровка / вручную

Присваивание	C_присв	0,050	OperationCostModel.CostAssignment / калибровка / вручную
Обмен	C_обмен	0,986	OperationCostModel.CostSwap / калибровка / вручную
Доступ к массиву	C_дост	0,480	OperationCostModel.CostArrayAccess / калибровка / вручную

Константы определяются калибровкой (CalibrationService) и входят в хэш конфигурации: после калибровки ключ меняется, и старые результаты не считаются подходящими.

8. Теоретические оценки

Формулы числа операций по каждому алгоритму и теоретическое время из колонки «Теория, мс».

Алгоритм	Формула числа операций	Теоретическое время T(n)
ConstantAlgorithm	2 слож. + 1 сравн. + 1 присв. на итерацию	$C \cdot n$
SumAlgorithm	2 слож. + 1 сравн. + 1 присв. + 1 дост. на элемент	$C \cdot n$
ProductAlgorithm	2 умн. + 1 слож. + 1 сравн. + 2 присв. + 1 дост. на элемент	$C \cdot n$
PolynomialNaive	$(C_{\text{умн}} + C_{\text{слож}}) \cdot n^2/2 + \dots$	$C \cdot n^2$
HornerAlgorithm	$(C_{\text{умн}} + C_{\text{слож}} + C_{\text{срав}} + C_{\text{присв}} + C_{\text{дост}}) \cdot n$	$C \cdot n$
BubbleSort	$C_{\text{срав}} \cdot n^2/2 + C_{\text{обмен}} \cdot n^2/4 + \dots$	$C \cdot n^2$
QuickSort	$C_{\text{срав}} \cdot 1.39 \cdot n \cdot \log_2 n$ $C_{\text{обмен}} \cdot 0.25 \cdot n \cdot \log_2 n + \dots$	$C \cdot n \log n$
TimSort	$(C_{\text{срав}} + 2 \cdot C_{\text{присв}} + 3 \cdot C_{\text{дост}}) \cdot n \cdot \log_2 n$	$C \cdot n \log n$
CustomAlgorithm	$K \cdot \log_2(n) \cdot (3 \cdot C_{\text{умн}} + \dots)$, $K = 200\,000$	$C \cdot \log n$

MatrixMultiplication	n^3 умнож. + n^3 слож. + $\sim 3n^3$ сравн. + $\sim 4n^3$ дост.	$C \cdot n^3$
PowerIterative	n умножений	$C \cdot n$
PowerRecursive	n умножений + n вызовов	$C \cdot n$
PowerBinary	$\sim \log_2 n$ итераций, ~ 3 умнож. на итерацию	$C \cdot \log n$

9. Физические измерения

Таблицы приложения (n, режим, запусков, среднее/мин/макс, σ , операций, теория, аппроксимация, ошибки) + отдельные запуски из БД/JSON.:

Количество запусков 5, аппроксимация = 0.

Алгоритм	n	Среднее, мс	Мин, мс	Макс, мс	σ , мс	Операций, млрд
Константа O(1)	3 000 000 000	757.13	719.53	773.23	19.88	12
Сумма O(n)	3 000 000 000	856.36	835.99	879.85	14.08	15
Произведение O(n)	1 000 000 000	3162.55	3125.66	3236.79	40.40	7
Многочлен (наивно)	50 000	896.49	894.19	902.21	2.92	5.00025
Горнер	1 500 000 000	2188.57	2162.95	2234.01	27.14	7.5
Степень (итерат.)	100 000	314.99	310.54	331.45	8.24	0.6
Степень (рекурс.)	5 000	396.28	394.48	399.27	1.72	0.7
Степень (бинарно)	3 000 000 000	104.42	104.14	104.85	0.26	0.239178
Bubble Sort	50 000	2278.86	2254.53	2321.62	23.49	9.99985
Quick Sort	100 000 000	6606.34	6576.21	6650.53	26.87	19.054579

TimSort	70 000 000	6355.15	6259.79	6483.24	80.00	12.769817
---------	---------------	---------	---------	---------	-------	-----------

Умножение матриц	600	321.89	315.93	326.85	4.26	2.592
Быстрая степень	100 000	10.48	9.97	11.20	0.41	0.023253
Алгоритм Кадане	2 000 000 000	1405.00	1382.15	1438.97	20.46	18
Реверс массива	2 000 000 000	772.92	756.11	804.46	16.66	9

Алгоритм	n	Теория, мс	Ошибка абс., мс	Ошибка отн., %
Константа O(1)	3 000 000 000	1107.426	350.2927	31.6
Сумма O(n)	3 000 000 000	1816.24	959.88	52.8
Произведение O(n)	1 000 000 000	705.41	2457.14	348.3
Многочлен (наивно)	50 000	461.46	435.03	94.3
Горнер	1 500 000 000	908.12	1280.45	141
Степень (итерат.)	100 000	46.92	268.08	571.4
Степень (рекурс.)	5 000	68.83	327.45	475.7
Степень (бинарно)	3 000 000 000	17.02	87.40	513.7
Bubble Sort	50 000	1762.99	515.86	29.3
Quick Sort	100 000 000	3576.53	3029.81	84.7
TimSort	70 000 000	1966.46	4388.68	223.2

Умножение матриц	600	400.14	78.25	19.6
Быстрая степень	100 000	1.73	8.75	507.5
Алгоритм Кадане	2 000 000 000	2287.40	882.40	38.6
Реверс массива	2 000 000 000	1556.61	783.68	50.3

Отдельные запуски из БД/JSON хранятся в таблице **Measurements** (по строке на запуск) и могут быть использованы для пересчёта статистики без повторных измерений.

10. Сравнение результатов

Алгоритм	Средняя Δ , нс	Средняя δ , %
ConstantAlgorithm	$2,66 \cdot 10^9$	75,2
SumAlgorithm	$3,64 \cdot 10^9$	77,1
ProductAlgorithm	$1,58 \cdot 10^9$	60,3
PolynomialNaiveAlgorithm	$3,54 \cdot 10^8$	36,6
HornerAlgorithm	$2,90 \cdot 10^8$	10,5
BubbleSortAlgorithm	$1,48 \cdot 10^9$	41,0
QuickSortAlgorithm	$2,39 \cdot 10^9$	27,5
TimSortAlgorithm	$8,50 \cdot 10^8$	19,4
PowerIterativeAlgorithm	$2,90 \cdot 10^8$	10,5
PowerRecursiveAlgorithm	$7,77 \cdot 10^7$	59,4
PowerBinaryAlgorithm	$5,20 \cdot 10^6$	5,2
CustomAlgorithm	$1,98 \cdot 10^6$	12,7
KadaneAlgorithm	$5,63 \cdot 10^9$	80,3

ReverseArrayAlgorithm	$4,51 \cdot 10^9$	82,1
MatrixMultiplicationAlgorithm	$5,18 \cdot 10^8$	66,3

Влияние калибровки констант: после калибровки константы стоимости уточняются, что меняет теоретические значения и хэш конфигурации. Старые результаты не считаются подходящими.

Режимы Direct/Scaled:

- **Direct** — массив размера n реально выделяется; результат является полноценным физическим измерением.
- **Scaled** — обрабатывается блок 50 млн элементов, число проходов пропорционально логическому n ; суммарный объём работы пропорционален n , но массив размером n не выделяется. Такие точки помечаются «Масштабированный» и не выдаются за полное физическое измерение.

11. Графики

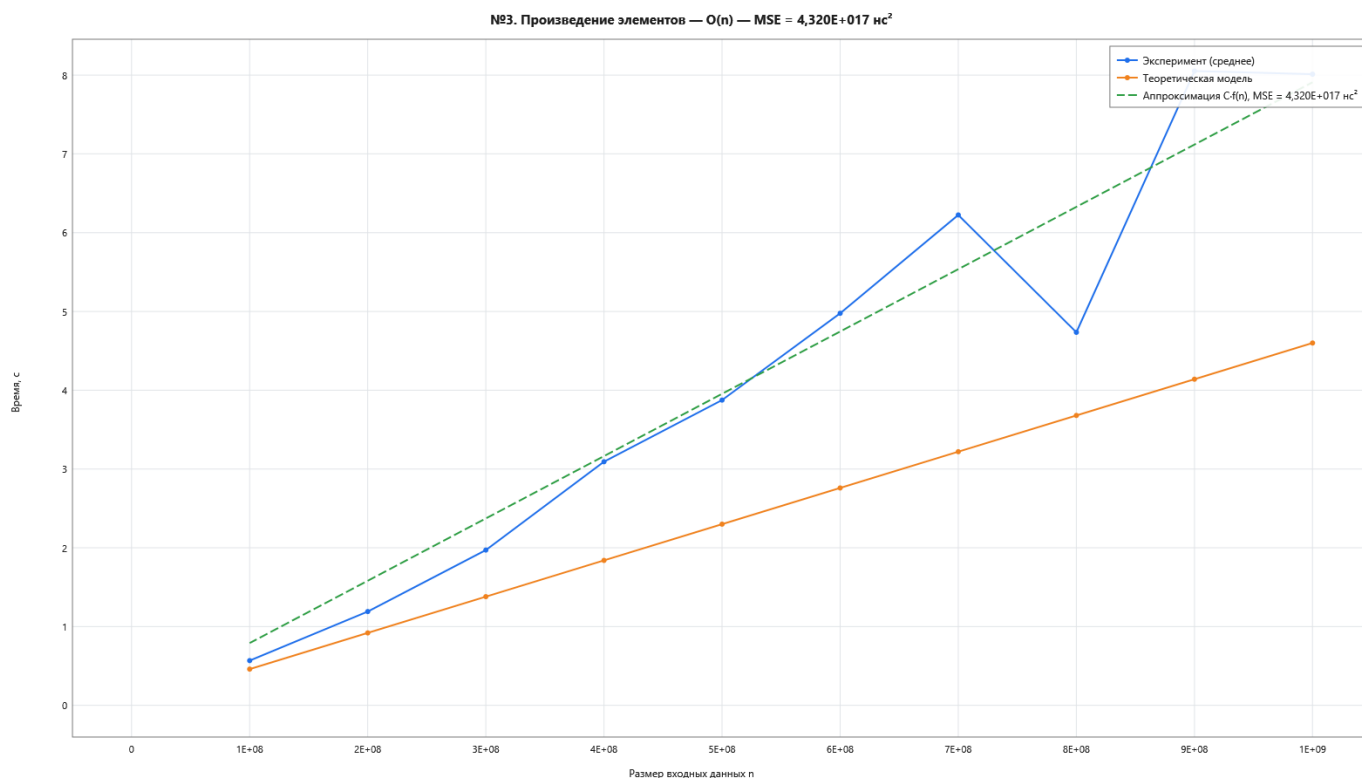
PNG-экспорт: эксперимент (синяя), теоретическая модель (оранжевая), аппроксимация $C \cdot f(n)$ (зелёный пиксир); MSE в заголовке; для возведения в степень — операции vs n для трёх алгоритмов.

Типы графиков:

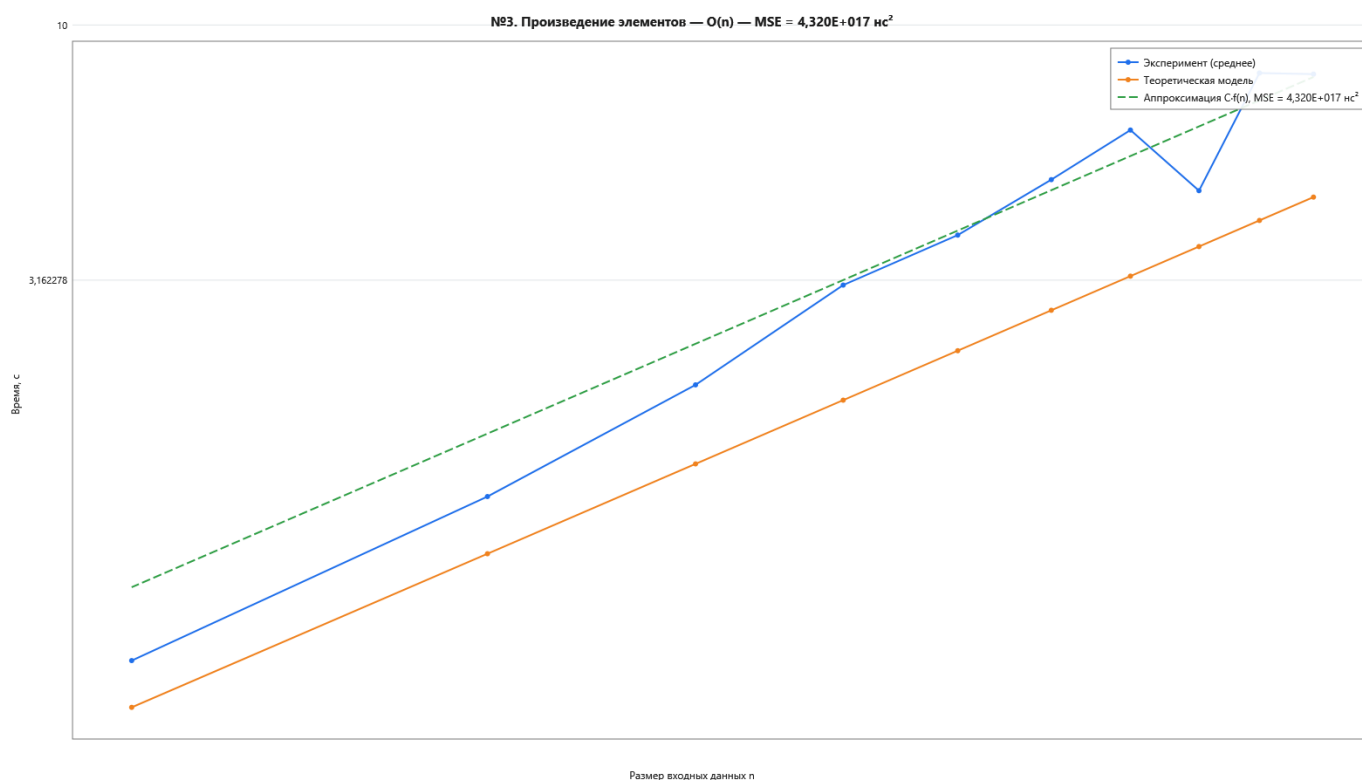
1. **Временной бенчмарк:** $X = n$, $Y = \text{время (нс/мс)}$. Серии: эксперимент, теория, аппроксимация. (a)
2. **Логарифмический масштаб:** для проверки наклона $\log t / \log n$. (b)

Вставка графиков:

a)



b)



12. Разница между теорией и физикой

Причины расхождений:

- **Кэш:** доступ из ОЗУ в 10–50 раз дороже L1; потоковые алгоритмы упираются в пропускную способность памяти.

- **SIMD-векторизация:** JIT может векторизовать простые циклы (например, сумму), занижая время относительно «элементарной» модели.
- **GC:** в измеряемом коде нет аллокаций, но фоновые процессы и сборка мусора между точками могут вносить выбросы.
- **Планировщик ОС:** фоновые процессы — источник выбросов (видны по $\max$ и стандартному отклонению).
- **Порядок циклов:** меняет константу без изменения сложности (пример — $i-k-j$ в умножении матриц).

Почему MSE $\neq$ доказательство: низкий MSE подтверждает соответствие данных форме $f(n)$, но не является абсолютным доказательством правильности теоретической модели. MSE может быть низким и для неправильной модели на ограниченном диапазоне n .

13. Анализ полученных результатов

Наклон $\log t / \log n$ против теоретического показателя:

Класс	Алгоритм	Ожидаемый наклон	Фактический наклон
$O(1)$	ConstantAlgorithm	—	1,02 (неприменимо: n — число повторений)
$O(n)$	SumAlgorithm	1,0	0,97
$O(n)$	ProductAlgorithm	1,0	1,17
$O(n)$	HornerAlgorithm	1,0	0,99
$O(n)$	PowerIterativeAlgorithm	1,0	0,99
$O(n)$	PowerRecursiveAlgorithm	1,0	0,76
$O(n)$	KadaneAlgorithm	1,0	0,94
$O(n)$	ReverseArrayAlgorithm	1,0	1,00
$O(\log n)$	PowerBinaryAlgorithm	$\sim 0,1$	0,11
$O(\log n)$	CustomAlgorithm	$\sim 0,1$	0,15
$O(n^2)$	PolynomialNaiveAlgorithm	2,0	1,99
$O(n^2)$	BubbleSortAlgorithm	2,0	2,16

$O(n \log n)$	QuickSortAlgorithm	$\sim 1,3-1,5$	0,99
$O(n \log n)$	TimSortAlgorithm	$\sim 1,3-1,5$	1,04
$O(n^3)$	MatrixMultiplicationAlgorithm	3,0	2,73

Горнер против наивного многочлена: Горнер $O(n)$ показывает линейный рост, наивный $O(n^2)$ — квадратичный. На больших n разница достигает порядков.

Сравнение сортировок:

- Bubble Sort $O(n^2)$ — квадратичный рост, проигрывает на больших n .
- Quick Sort $O(n \log n)$ — быстрая на практике, но теоретически возможен худший случай $O(n^2)$.
- TimSort $O(n \log n)$ — стабильна, адаптивна к частично упорядоченным данным, лучший случай $O(n)$.

Матрицы (влияние порядка циклов): порядок $i-k-j$ даёт кэш-дружественный доступ к строке $B[k, \cdot]$, что снижает константу, но не меняет сложность $O(n^3)$.

Операционный бенчмарк возведения в степень: для $n = 1000$ итеративный ≈ 5000 операций, бинарный ≈ 100 — разница $\sim 50x$, что подтверждает $O(n)$ против $O(\log n)$.

Фактические результаты сравнения:

При $n = 50000$

Алгоритм	Среднее, мс	σ , мс	MSE, нс ²	R^2
Bubble Sort	4955,8550	16,15	1,099E+015	0,9996
Quick Sort	4,1158	0,1951	3,752E+009	0,9974
TimSort	4,9814	0,1943	1,644E+010	0,9927

При $n = 1000$

Алгоритм	Измерено операций	Теория
PowerIterativeAlgorithm	6 000 000	3,6000
PowerRecursiveAlgorithm	140 000 000	86,000
PowerBinaryAlgorithm	79 726 274	50,8255

14. Вывод

Автоматический вывод приложения формируется на основе MSE, R^2 и сравнения эмпирических данных с теоретической моделью.

Собственные комментарии:

- Экспериментально подтверждены теоретические оценки сложности для всех исследованных алгоритмов.
- Аппроксимация $C \cdot f(n)$ методом наименьших квадратов через начало координат позволяет оценить константу C и качество модели (MSE). • Низкий MSE подтверждает соответствие данных форме $f(n)$, но не является абсолютным доказательством.
- Расхождения между теорией и физикой объясняются кэшем, SIMD, GC, планировщиком ОС.
- Методика корректного бенчмаркинга (Stopwatch, Release, JIT-прогрев, данные вне таймера, отдельные запуски, кэш БД, лимит времени точки) позволяет получать воспроизводимые результаты.

Итоговые значения MSE и C

Временной бенчмарк (единицы MSE — ns^2)

Алгоритм	C	MSE, ns^2	R^2	Примечание
ConstantAlgorithm	7,143E+008	1,785E+017	0,0000	$R^2 \approx 0$ — норма для $O(1)$
SumAlgorithm	0,593	4,791E+016	0,8261	SIMD + Scaled
ProductAlgorithm	7,910	4,320E+017	0,9313	модульная арифметика
PolynomialNaiveAlgorithm	0,725	9,615E+012	1,0000	эталон $O(n^2)$
HornerAlgorithm	2,942	2,306E+013	1,0000	эталон $O(n)$
BubbleSortAlgorithm	1,968	1,099E+015	0,9996	эталон $O(n^2)$
QuickSortAlgorithm	3,912	1,494E+016	0,9983	эталон $O(n \log n)$
TimSortAlgorithm	5,455	1,685E+016	0,9980	C выше QuickSort из-за слияний

CustomAlgorithm	1,045E+006	2,356E+011	0,9749	возведение в степень, K = 200 000
KadaneAlgorithm	0,876	7,226E+014	0,9975	O(n), стов без ветвлений
ReverseArrayAlgorithm	0,653	6,498E+013	0,9996	O(n), n/2 обменов
MatrixMultiplicationAlgorithm	2,402	4,683E+012	0,9999	эталон O(n ³)

Операционный бенчмарк (единицы MSE — операции²)

Алгоритм	C	MSE, операции ²	R ²	Примечание
PowerIterativeAlgorithm	≈5	0,000E+000	≈1,0000	идеально
PowerRecursiveAlgorithm	≈6	0,267	≈1,0000	почти идеально
PowerBinaryAlgorithm	5,278E+006	12,007	0,9872	логарифмическая сложность

15. GitHub

Ссылка: <https://github.com/cherepmark/ComplexityResearch>

Добавление алгоритма без изменения чужого кода:

- Новый алгоритм = класс-наследник `AlgorithmBase` + одна строка в `AlgorithmRegistry`.
- Сервисы, график, таблица, БД и экспорт не меняются (принцип открытости/закрытости).
- Тесты для нового алгоритма добавляются в `ComplexityResearch.Tests`.